

Merge Sort

Merge Sort is a highly efficient, comparison based, divide-and-conquer sorting algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element.

These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

Always $O(N \log N)$ in all cases (Best, Average, worst)

$O(n)$ due to the auxiliary array used during the merging phase

Week 3 Module:

Date: 29/06/2026

Topic 1: Merge Sort

Question 1 (Easy) – Rank Students by Marks

A school has recently conducted a scholarship examination for all students of Class 12. The examination department has collected the marks of all students in an array. Due to the large number of participants, manually arranging the marks in ascending order has become difficult.

Your task is to write a program that takes the marks of N students and sorts them in ascending order using the Merge Sort algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element. These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

The school administration wants the marks to be displayed in increasing order so they can easily identify the lowest and highest scoring students and generate performance reports.

Input

- First line contains an integer N , representing the number of students.
- Second line contains N space-separated integers, representing the marks of the students.

Output

- Print the marks in ascending order.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{Marks} \leq 100$

Example

Input:

6
78 45 92 34 67 81

Output:

34 45 67 78 81 92

Rank students by marks

```
// include <stdio.h>
```

```
// function to merge two halves of an array
```

```
void merge (int array-arr [], int left, int mid,  
            int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
// Temporary arrays
```

```
int L[n1], R[n2];
```

```
// copy data to temporary arrays L[] and R[]
```

```
for (int i = 0; i < n1; i++)
```

```
    L[i] = array[left + i];
```

```
for (int j = 0; j < n2; j++)
```

```
    R[j] = array[mid + 1 + j];
```

```
int i = 0, j = 0, k = left;
```

```
// merge the temporary arrays back into  
array[left..right]
```

```
while (i < n1 && j < n2) {
```

```
    if (L[i] <= R[j]) {
```

```
        array[k] = L[i];
```

```
        i++;
```

```
    } else {
```

```
        array[k] = R[j];
```

```
        j++;
```

```
    }
```

```
    k++;
```

```
}
```

// Copy the remaining elements of L[], if there are any

```
while (i < n1) {  
    arr[k] = L[i];  
    i++;  
    k++;  
}
```

// Copy the remaining elements of R[], if there are any

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;  
}
```

```
}}
```

// Main function to perform merge sort

```
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {
```

// Calculate mid point to avoid overflow

```
int mid = left + (right - left) / 2;
```

// Sort first and second halves

```
mergeSort(arr, left, mid);
```

```
mergeSort(arr, mid + 1, right);
```

```
// merge the sorted halves  
merge(arr, left, mid, right);  
}}
```

```
int main() {  
    int N;  
    scanf("%d", &N); // Number of students  
    int marks[N];  
    for (int i=0; i<N; i++) {  
        scanf("%d", &marks[i]);  
    }  
    mergeSort(marks, 0, N-1);  
  
    for (int i=0; i<N; i++) {  
        printf("%d", marks[i]);  
    }  
    printf("\n");  
    return 0;  
}
```

I/p: 6 78 45 92 34 67 81

O/p: 34 45 67 78 81 92

```
23     arr[k] = L[i];
24     i++;
25 } else {
26     arr[k] = R[j];
27     j++;
28 }
29 k++;
30 }
31
32 // Copy left side
33 while (i < n1) {
34     arr[k] = L[i];
35     i++;
36     k++;
37 }
38
39 // Copy right side
40 while (j < n2) {
41     arr[k] = R[j];
42     j++;
43     k++;
44 }
45 }
46
```

```
#include <stdio.h>
```

```
// Merge the two parts together
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    // Temporary arrays
```

```
    int L[n1], R[n2];
```

```
    // Put values into temp arrays
```

```
    for (int i = 0; i < n1; i++)
```

```
    |     L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++)
```

```
    |     R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    // Compare and merge
```

```
    while (i < n1 && j < n2) {
```

```
    |     if (L[i] <= R[j]) {
```

```
    |         arr[k] = L[i];
```

```
    |         i++;
```



```
45 }  
46  
47 // Merge Sort function  
48 void mergeSort(int arr[], int left, int right) {  
49     if (left < right) {  
50  
51         // Find the middle  
52         int mid = left + (right - left) / 2;  
53  
54         // Sort both halves  
55         mergeSort(arr, left, mid);  
56         mergeSort(arr, mid + 1, right);  
57  
58         // Merge them back  
59         merge(arr, left, mid, right);  
60     }  
61 }  
62  
63 int main() {  
64     int N;  
65     scanf("%d", &N);    // Total marks to enter  
66  
67     int marks[N];  
68
```




```
61 }  
62  
63 int main() {  
64     int N;  
65     scanf("%d", &N);    // Total marks to enter  
66  
67     int marks[N];  
68  
69     // Read all marks  
70     for (int i = 0; i < N; i++) {  
71         scanf("%d", &marks[i]);  
72     }  
73  
74     mergeSort(marks, 0, N - 1);  
75  
76     // Print sorted marks  
77     for (int i = 0; i < N; i++) {  
78         printf("%d ", marks[i]);  
79     }  
80  
81     printf("\n");  
82  
83     return 0;  
84 }
```

Output

Clear

```
6 78 45 92 34 67 81  
34 45 67 78 81 92
```

```
=== Code Execution Successful ===
```

Question 2 (Easy) – Organizing Product Prices

An e-commerce company stores product prices in the order they are added to the platform. Before publishing a seasonal sales report, the company wants all product prices to be arranged in ascending order.

Since the number of products can be very large, an efficient sorting technique is required. Your task is to implement Merge Sort to arrange the prices from the cheapest product to the most expensive product.

The program should divide the list into smaller parts, sort each part, and merge them back together while maintaining the correct order.

Input

- First line contains integer N .
- Second line contains N space-separated product prices.

Output

- Print the sorted prices in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Price} \leq 1000000$

Example

Input:

5
2500 1500 4000 3000 1000

Output:

1000 1500 2500 3000 4000

Q.2 Organizing Product prices

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
Void merge (long long arr[], int left, int mid,  
            int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    long long L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k++] = L[i++];
```

```
        }
```

```
        else {
```

```
            arr[k++] = R[j++];
```

```
        } }
```

```
    while (i < n1) arr[k++] = L[i++];
```

```
    while (j < n2) arr[k++] = R[j++];
```

```
}
```

```
Void mergeSort (long long arr[], int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;
```

```
        mergeSort (arr, left, mid);
```

```
mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}}
```

```
int main() {  
    int N;  
    scanf("%d", &N);
```

```
// Using dynamically allocated array  
// for large & considerable  
long long *price = (long long *) malloc  
    (N * sizeof(long long));
```

```
for (int i = 0; i < N; i++) {  
    scanf("%lld", &price[i]);  
}
```

```
mergeSort(price, 0, N - 1);
```

```
for (int i = 0; i < N; i++) {  
    printf("%lld", price[i]);  
}
```

```
printf("\n");  
free(price);  
return 0;  
}
```

I/p:- 5 2500 1500 4000 3000 1000

O/p:- 1000 1500 2500 3000 4000



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Merge two sorted parts
5  void merge(long long arr[], int left, int mid, int right) {
6      int n1 = mid - left + 1;
7      int n2 = right - mid;
8
9      long long L[n1], R[n2];
10
11     // Copy values into temp arrays
12     for (int i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14
15     for (int j = 0; j < n2; j++)
16         R[j] = arr[mid + 1 + j];
17
18     int i = 0, j = 0, k = left;
19
20     // Merge both arrays
21     while (i < n1 && j < n2) {
22         if (L[i] <= R[j]) {
23             arr[k++] = L[i++];
24         } else {
25             arr[k++] = R[j++];
26         }
27     }
28     while (i < n1) arr[k++] = L[i++];
29     while (j < n2) arr[k++] = R[j++];
30 }
```

```
|     arr[k++] = L[i++];  
| } else {  
|     arr[k++] = R[j++];  
| }  
}
```

```
// Copy remaining values
```

```
while (i < n1)
```

```
|     arr[k++] = L[i++];
```

```
while (j < n2)
```

```
|     arr[k++] = R[j++];
```

```
}
```

```
// Merge Sort
```

```
void mergeSort(long long arr[], int left, int right) {
```

```
    if (left < right) {
```

```
        // Find middle
```

```
        int mid = left + (right - left) / 2;
```

```
        // Sort both halves
```

```
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```



```
int main() {  
    int N;  
    scanf("%d", &N);  
  
    // Allocate memory for numbers  
    long long *prices = (long long *)malloc(N * sizeof(long long));  
  
    // Read the values  
    for (int i = 0; i < N; i++) {  
        scanf("%lld", &prices[i]);  
    }  
  
    mergeSort(prices, 0, N - 1);  
  
    // Print sorted values  
    for (int i = 0; i < N; i++) {  
        printf("%lld ", prices[i]);  
    }  
  
    printf("\n");  
  
    free(prices);  
    return 0;  
}
```



```
// Sort both halves
```

```
mergeSort(arr, left, mid);
```

```
mergeSort(arr, mid + 1, right);
```

```
// Merge them
```

```
merge(arr, left, mid, right);
```

```
}
```

```
}
```

```
int main() {
```

```
    int N;
```

```
    scanf("%d", &N);
```

```
    // Allocate memory for numbers
```

```
    long long *prices = (long long *)malloc(N * sizeof(long long));
```

```
    // Read the values
```

```
    for (int i = 0; i < N; i++) {
```

```
        scanf("%lld", &prices[i]);
```

```
    }
```

```
    mergeSort(prices, 0, N - 1);
```

```
    // Print sorted values
```

```
    for (int i = 0; i < N; i++) {
```

Output

Clear

```
5 2500 1500 4000 3000 1000  
1000 1500 2500 3000 4000
```

```
=== Code Execution Successful ===
```

Question 3 (Medium) – Sort Employee Records by Salary

A multinational company has employee salary records stored in an array. Before generating annual reports, the Human Resources department wants the salary records arranged in ascending order.

The company expects the number of employees to grow significantly over time, so an efficient sorting algorithm must be used. You are required to implement Merge Sort and display the salaries in increasing order.

Additionally, after sorting, the company wants to know:

1. The lowest salary.
2. The highest salary.
3. The difference between the highest and lowest salary.

Input

- First line contains integer N .
- Second line contains N space-separated salaries.

Output

- First line: Sorted salaries.
- Second line: Lowest salary.
- Third line: Highest salary.
- Fourth line: Salary difference.

Constraints

- $1 \leq N \leq 100000$
- $1000 \leq \text{Salary} \leq 1000000$

Example

Input:

```
7
45000 25000 70000 50000 30000 90000 60000
```

Output:

```
25000 30000 45000 50000 60000 70000 90000
Lowest Salary: 25000
Highest Salary: 90000
Difference: 65000
```

Q.3 Sort Employee records by salary

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
Void merge (int arr[], int left, int mid, int right) {  
    int n1 = mid - left + 1, n2 = right - mid;  
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];  
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) arr[k++] = L[i++];  
        else arr[k++] = R[j++];  
    }
```

```
    while (i < n1) arr[k++] = L[i++];  
    while (j < n2) arr[k++] = R[j++];  
}
```

```
Void mergeSort (int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort (arr, left, mid);  
        mergeSort (arr, mid + 1, right);  
        merge (arr, left, mid, right);  
    }
```

```

int main(){
int N;
scanf("%d", &N);
int *salaries = (int *) malloc (N * sizeof(int));
for (int i = 0; i < N; i++)
    scanf("%d", &salaries[i]);
}
mergeSort(salaries, 0, N-1);

// Sorted Salaries
for (int i = 0; i < N; i++) {
    printf("%d", salaries[i]);
}
printf("\n");

12. Lowest, Highest and Difference
printf("Lowest Salary : %d\n", salaries[0]);
printf("Highest Salary : %d\n", salaries[N-1]);
printf("Difference : %d\n", salaries[N-1] - salaries[0]);

free(salaries);
return 0;
}

```

I/p:- 7 45000 25000 70000 50000 30000 90000
60000

O/p:- 25000 30000 45000 50000 60000 70000 90000
Lowest Salary : 25000 Highest Salary : 90000
Difference : 65000



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Merge two sorted parts
5 void merge(int arr[], int left, int mid, int right) {
6     int n1 = mid - left + 1;
7     int n2 = right - mid;
8
9     int L[n1], R[n2];
10
11     // Copy values
12     for (int i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14
15     for (int j = 0; j < n2; j++)
16         R[j] = arr[mid + 1 + j];
17
18     int i = 0, j = 0, k = left;
19
20     // Merge both arrays
21     while (i < n1 && j < n2) {
22         if (L[i] <= R[j])
23             arr[k++] = L[i++];
24         else
```



```
|     arr[k++] = L[i++];  
|     else  
|     arr[k++] = R[j++];  
| }
```

```
// Copy remaining values
```

```
while (i < n1)
```

```
|     arr[k++] = L[i++];
```

```
while (j < n2)
```

```
|     arr[k++] = R[j++];
```

```
}
```

```
// Merge Sort
```

```
void mergeSort(int arr[], int left, int right) {
```

```
    if (left < right) {
```

```
|     // Find middle
```

```
|     int mid = left + (right - left) / 2;
```

```
|     // Sort both halves
```

```
|     mergeSort(arr, left, mid);
```

```
|     mergeSort(arr, mid + 1, right);
```




```
mergeSort(arr, left, mid);  
mergeSort(arr, mid + 1, right);
```

```
// Merge them  
merge(arr, left, mid, right);  
}
```

```
}
```

```
int main() {
```

```
int N;
```

```
scanf("%d", &N);
```

```
// Allocate memory
```

```
int *salaries = (int *)malloc(N * sizeof(int));
```

```
// Read salaries
```

```
for (int i = 0; i < N; i++) {
```

```
    scanf("%d", &salaries[i]);
```

```
}
```

```
mergeSort(salaries, 0, N - 1);
```

```
// Print sorted salaries
```

```
for (int i = 0; i < N; i++) {
```

```
    printf("%d ", salaries[i]);
```

```
// Allocate memory
int *salaries = (int *)malloc(N * sizeof(int));

// Read salaries
for (int i = 0; i < N; i++) {
    scanf("%d", &salaries[i]);
}

mergeSort(salaries, 0, N - 1);

// Print sorted salaries
for (int i = 0; i < N; i++) {
    printf("%d ", salaries[i]);
}
printf("\n");

// Print lowest, highest and difference
printf("Lowest Salary: %d\n", salaries[0]);
printf("Highest Salary: %d\n", salaries[N - 1]);
printf("Difference: %d\n", salaries[N - 1] - salaries[0]);

free(salaries);
return 0;
```

```
}
```

Output

[Clear](#)

```
7 45000 25000 70000 50000 30000 90000 60000
```

```
25000 30000 45000 50000 60000 70000 90000
```

```
Lowest Salary: 25000
```

```
Highest Salary: 90000
```

```
Difference: 65000
```

```
=== Code Execution Successful ===
```

Quick sort

Quick sort is a fast, highly optimized, divide and conquers sorting algorithm.

It works by picking a 'pivot' element from the array and partitioning the other elements into two sub-arrays (according to whether they are less than or greater than the pivot).

Average Case is $O(N \log N)$. worst case is $O(N^2)$ (occurs when the array is already sorted, and the pivot chosen is always the extreme element)

$O(\log N)$ due to recursive stack space.
Quick sort is an in-place sorting algorithm.

Topic 2: Quick Sort

Question 1 (Easy) – Race Timing Analysis

A sports academy records the completion times (in seconds) of runners participating in a race. The coach wants to arrange the timings from the fastest runner to the slowest runner.

To process large datasets efficiently, the academy has decided to use the Quick Sort algorithm. Your task is to sort the race timings in ascending order.

Input

- First line contains integer N .
- Second line contains N space-separated timings.

Output

- Print the timings in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 10000$

Example

Input:

```
6
55 49 62 58 45 51
```

Output:

```
45 49 51 55 58 62
```

1) Race Timing Analysis

```
#include <stdio.h>
```

```
// function to swap two elements
```

```
void swap(int *a, int *b) {
```

```
    int t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
// Partition function for Quick Sort
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high]; // Choosing the last element  
                           as pivot
```

```
    int i = (low - 1); // Index of smallest element
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        // If current element is smallest than the pivot
```

```
        if (arr[j] < pivot) {
```

```
            i++;
```

```
            swap(arr[i], arr[j]);
```

```
        }
```

```
    }
```

```
    swap(arr[i + 1], arr[high]);
```

```
    return (i + 1);
```

```
}
```

```
// Main Quick Sort function
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

// partition index

int pi = partition (arr, low, high);

// recursively sort elements before and after partition

quickSort (arr, low, pi-1);

quickSort (arr, pi+1, high);

}}

int main () {

int N;

scanf ("%d", &N);

int timings[N];

for (int i = 0; i < N; i++) {

scanf ("%d", &timings[i]);

}

quickSort (timings, 0, N-1);

for (int i = 0; i < N; i++) {

printf ("%d", timings[i]);

}

printf ("\n");

return 0;

}

I/p:- 6 55 49 62 58 45 51

O/p:- 45 49 51 55 58 62

```
1  #include <stdio.h>
2
3  // Swap two numbers
4  void swap(int *a, int *b) {
5      int t = *a;
6      *a = *b;
7      *b = t;
8  }
9
10 // Find the correct position of pivot
11 int partition(int arr[], int low, int high) {
12
13     // Last element is pivot
14     int pivot = arr[high];
15     int i = low - 1;
16
17     // Put smaller elements on left
18     for (int j = low; j < high; j++) {
19         if (arr[j] < pivot) {
20             i++;
21             swap(&arr[i], &arr[j]);
22         }
23     }
24
25     // i is the correct position of pivot
26     swap(&arr[i], &arr[high]);
27     return i;
28 }
```



```
23     }
24
25     swap(&arr[i + 1], &arr[high]);
26     return i + 1;
27 }
28
29 // Quick Sort
30 void quickSort(int arr[], int low, int high) {
31     if (low < high) {
32
33         // Get pivot position
34         int pi = partition(arr, low, high);
35
36         // Sort left and right
37         quickSort(arr, low, pi - 1);
38         quickSort(arr, pi + 1, high);
39     }
40 }
41
42 int main() {
43     int N;
44     scanf("%d", &N);
45
46     int timings[N];
```



```
40 }
41
42 int main() {
43     int N;
44     scanf("%d", &N);
45
46     int timings[N];
47
48     // Read timings
49     for (int i = 0; i < N; i++) {
50         scanf("%d", &timings[i]);
51     }
52
53     quickSort(timings, 0, N - 1);
54
55     // Print sorted timings
56     for (int i = 0; i < N; i++) {
57         printf("%d ", timings[i]);
58     }
59
60     printf("\n");
61
62     return 0;
63 }
```

Output

Clear

```
6 55 49 62 58 45 51  
45 49 51 55 58 62
```

```
=== Code Execution Successful ===
```

Question 2 (Easy) – Organizing Library Book IDs

A library stores books using unique identification numbers. Due to years of additions and removals, the IDs are no longer arranged properly.

The librarian wants all IDs sorted in ascending order before migrating the data to a new management system. Implement Quick Sort to arrange the IDs efficiently.

Input

- First line contains integer N .
- Second line contains N space-separated book IDs.

Output

- Print the sorted IDs.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq ID \leq 10^9$

Example

Input:

```
5
1025 1001 1050 1010 1005
```

Output:

```
1001 1005 1010 1025 1050
```

Q.2

Organizing Library Book IDs

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
Void swap (long long *a, long long *b) {  
    long long t = *a;  
    *a = *b;  
    *b = t;  
}
```

```
int partition (long long arr[], int low, int high) {  
    long long pivot = arr[high];  
    int i = (low - 1);  
    for (int j = low; j <= high - 1; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap (&arr[i], &arr[j]);  
        }  
    }  
    swap (&arr[i+1], &arr[high]);  
    return (i+1);  
}
```

```
Void quickSort (long long arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition (arr, low, high);  
        quickSort (arr, low, pi - 1);  
        quickSort (arr, pi + 1, high);  
    }  
}
```

```

int main() {
    int N;
    scanf("%d", &N);

    // Program to read N book IDs (max  $10^9$ ) for safety
    long long *bookIDs = (long long *) malloc
        (N * sizeof(long long));
    for (int i = 0; i < N; i++) {
        scanf("%lld", &bookIDs[i]);
    }

    quickSort (bookIDs, 0, N-1);

    for (int i = 0; i < N; i++) {
        printf("%lld", bookIDs[i]);
    }
    printf("\n");
    free (bookIDs);
    return 0;
}

```

I/p:- 5 1025 1001 1050 1010 1005

O/p:- 1001 1005 1010 1025 1050



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Swap two numbers
5  void swap(long long *a, long long *b) {
6      long long t = *a;
7      *a = *b;
8      *b = t;
9  }
10
11 // Find the pivot position
12 int partition(long long arr[], int low, int high) {
13     long long pivot = arr[high];
14     int i = low - 1;
15
16     // Put smaller values on left
17     for (int j = low; j < high; j++) {
18         if (arr[j] < pivot) {
19             i++;
20             swap(&arr[i], &arr[j]);
21         }
22     }
23
24     swap(&arr[i + 1], &arr[high]);
25 }
```

```
int main() {  
    int N;  
    scanf("%d", &N);  
  
    // Allocate memory  
    long long *bookIDs = (long long *)malloc(N * sizeof(long long));  
  
    // Read book IDs  
    for (int i = 0; i < N; i++) {  
        scanf("%lld", &bookIDs[i]);  
    }  
  
    quickSort(bookIDs, 0, N - 1);  
  
    // Print sorted IDs  
    for (int i = 0; i < N; i++) {  
        printf("%lld ", bookIDs[i]);  
    }  
  
    printf("\n");  
  
    free(bookIDs);  
    return 0;  
}
```



```
23
24     swap(&arr[i + 1], &arr[high]);
25     return i + 1;
26 }
27
28 // Quick Sort
29 void quickSort(long long arr[], int low, int high) {
30     if (low < high) {
31
32         // Get pivot position
33         int pi = partition(arr, low, high);
34
35         // Sort left and right
36         quickSort(arr, low, pi - 1);
37         quickSort(arr, pi + 1, high);
38     }
39 }
40
41 int main() {
42     int N;
43     scanf("%d", &N);
44
45     // Allocate memory
46     long long *bookIDs = (long long *)malloc(N * sizeof(long long));
```

Output

Clear

```
5 1025 1001 1050 1010 1005  
1001 1005 1010 1025 1050
```

```
=== Code Execution Successful ===
```

Question 3 (Medium) – Online Store Sales Ranking

An online marketplace tracks the daily sales made by different sellers. At the end of each week, management wants to identify the best-performing sellers.

You are given the total sales made by N sellers. Using Quick Sort, arrange the sales figures in descending order.

After sorting, display:

1. Sorted sales figures.
2. Top 3 highest sales values.
3. Average of the top 3 sales values.

Input

- First line contains integer N .
- Second line contains N space-separated sales values.

Output

- Sorted sales figures in descending order.
- Top 3 sales values.
- Average of top 3 sales.

Constraints

- $3 \leq N \leq 100000$
- $1 \leq \text{Sales} \leq 1000000$

Example

Input:

```
6
1200 5000 3000 2500 7000 4500
```

Output:

```
7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500
```

Q. 3

Online Store Sales Ranking

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
Void swap(long long *a, long long *b) {  
    long long t = *a;  
    *a = *b;  
    *b = t;  
}
```

// Modified partition function for Descending
orders

```
int partitionDescending(long long arr[], int low,  
    int high) {  
    long long pivot = arr[high];  
    int i = (low-1);  
    for (int j = low ; j <= high - 1 ; j++) {  
        // Change logic to greater than for descending  
        if (arr[j] > pivot) {  
            i++;  
            swap(&arr[i], &arr[j]);  
        }  
    }  
    swap(&arr[i+1], &arr[high]);  
    return (i+1);  
}
```

```
Void quickSortDescending(long long arr[], int low,  
    int high) {  
    if (low < high) {
```

```
int pi = partitionDescending(arr, low, high);  
quickSortDescending(arr, low, pi - 1);  
quickSortDescending(arr, pi + 1, high);  
}
```

```
int main() {  
    int N;  
    scanf("%d", &N);
```

```
    long long *sales = (long long *) malloc (N * sizeof(long long))
```

```
    for (int i = 0; i < N; i++) {  
        scanf("%lld", &sales[i]);  
    }
```

```
// Sort in Descending orders  
quickSortDescending(sales, 0, N - 1);
```

```
// 1. Print sorted sales  
for (int i = 0; i < N; i++) {  
    printf("%lld", sales[i]);  
}  
printf("\n");
```

```
// 2. Print Top 3  
printf("Top 3: ");  
for (int i = 0; i < 3 && i < N; i++) {  
    printf("%lld", sales[i]);  
}  
printf("\n");
```

// 3. Compute average of top 3

```
long long sum = 0;
int count = (N < 3) ? N : 3;
for (int i = 0; i < count; i++) {
    sum += sales[i];
}
printf("Average : %.2lld\n", sum / count);
free(sales);
return 0;
}
```

I/p:-

6 1200 5000 3000 2500 7000 4500

O/p:-

7000 5000 4500 3000 2000 1200

Top 3: 7000 5000 4500

Average: 5500



```
#include <stdio.h>
#include <stdlib.h>

// Swap two numbers
void swap(long long *a, long long *b) {
    long long t = *a;
    *a = *b;
    *b = t;
}

// Find pivot position
int partitionDescending(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;

    // Put bigger values on left
    for (int j = low; j < high; j++) {
        if (arr[j] > pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
}
```

```
23
24     swap(&arr[i + 1], &arr[high]);
25     return i + 1;
26 }
27
28 // Quick Sort in descending order
29 void quickSortDescending(long long arr[], int low, int high) {
30     if (low < high) {
31
32         // Get pivot position
33         int pi = partitionDescending(arr, low, high);
34
35         // Sort left and right
36         quickSortDescending(arr, low, pi - 1);
37         quickSortDescending(arr, pi + 1, high);
38     }
39 }
40
41 int main() {
42     int N;
43     scanf("%d", &N);
44
45     // Allocate memory
46     long long *sales = (long long *)malloc(N * sizeof(long long));
```



```
57     printf("%lld ", sales[i]);
58 }
59 printf("\n");
60
61 // Print top 3
62 printf("Top 3: ");
63 for (int i = 0; i < 3 && i < N; i++) {
64     printf("%lld ", sales[i]);
65 }
66 printf("\n");
67
68 // Find average of top 3
69 long long sum = 0;
70 int count = (N >= 3) ? 3 : N;
71
72 for (int i = 0; i < count; i++) {
73     sum += sales[i];
74 }
75
76 printf("Average: %lld\n", sum / count);
77
78 free(sales);
79 return 0;
80 }
```

```
// Allocate memory
long long *sales = (long long *)malloc(N * sizeof(long long));

// Read sales
for (int i = 0; i < N; i++) {
    scanf("%lld", &sales[i]);
}

quickSortDescending(sales, 0, N - 1);

// Print sorted sales
for (int i = 0; i < N; i++) {
    printf("%lld ", sales[i]);
}
printf("\n");

// Print top 3
printf("Top 3: ");
for (int i = 0; i < 3 && i < N; i++) {
    printf("%lld ", sales[i]);
}
printf("\n");

// Find average of top 3
```

Output

Clear

6 1200 5000 3000 2500 7000 4500

7000 5000 4500 3000 2500 1200

Top 3: 7000 5000 4500

Average: 5500

=== Code Execution Successful ===